BLE-BASED PROXIMITY ATTENDANCE SYSTEM WITH BIOMETRIC VERIFICATION

A PROJECT REPORT

Submitted by

ADITYA KUMAR NATH RA2411003020203

ABHISHEK S RA2411003020208

GOVIND GOEL RA2411003020215

Under the guidance of

Dr. Lalitha K.

(Assistant Professor, Department of Electronics and Communication Engineering)

for the course 21ECO101T / Short Range Wireless Communication

in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI - 600089

OCT 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled "BLE-Based Proximity Attendance System with Biometric Verification" is the bonafide work of ADITYA KUMAR NATH [REG NO: RA2411003020203], ABHISHEK S [REG NO: RA2411003020208] and GOVIND GOEL [REG NO: RA2411003020215], who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr. Lalitha K.
Assistant Professor
Electronics and Communication Engineering,
SRM Institute of Science and Technology,
Ramapuram, Chennai.

SIGNATURE

Dr.K.Raja M.E., Ph.D.,
Professor and Head
Computer Science and Engineering,
SRM Institute of Science and Technology,
Ramapuram, Chennai.

Submitted for the project viva-voce held on _____ at SRM Institute of Science and Technology, Ramapuram, Chennai - 600089.

INTERNAL EXAMINER 1 **INTERNAL EXAMINER 2**

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI - 89

DECLARATION

We hereby declare that the entire work contained in this project report titled "BLE-BASED PROXIMITY ATTENDANCE SYSTEM WITH BIOMETRIC VERIFICATION" has been carried out by ADITYA KUMAR NATH [REG NO: RA2411003020203], ABHISHEK S [REG NO: RA2411003020208] and GOVIND GOEL [REG NO: RA2411003020215] at SRM Institute of Science and Technology, Ramapuram Campus, Chennai - 600089, under the guidance of Dr. Lalitha K., Assistant Professor, Department of Electronics and Communication Engineering.

Place: Chennai
Date:

ADITYA KUMAR NATH

ABHISHEK S

GOVIND GOEL

ABSTRACT

Classroom attendance is still taken mostly by roll call or a signed sheet, which costs several minutes of every class and is easy to fake: a student can answer for an absent friend. This project presents an attendance system that uses Bluetooth Low Energy (BLE), the short-range radio built into every modern smartphone, to mark only the students who are physically present. No extra hardware is required. The teacher and the students run the same Android application, written in Flutter, in two modes.

The teacher's phone broadcasts a random 32-bit session code in its BLE advertisements. A student's phone scans for this code, confirms the student with the phone's fingerprint or face unlock, and then advertises a 24-byte payload carrying the session code, a per-install phone ID and the roll number. The teacher's phone accepts the broadcast only if the session code matches, the roll number is on the class list, the phone is the one first registered for that roll number, and the Received Signal Strength Indicator (RSSI) is at or above an adjustable cutoff. It then returns the roll number in its own advertisement as an acknowledgement, and the student's phone shows "Present marked". The whole exchange is connectionless: no pairing is needed, so one teacher phone can serve a whole class.

The system was tested end to end with an Android 16 phone in student mode and a Windows laptop's Bluetooth adapter acting as the teacher through a Python script that uses the same packet format, and the teacher mode was exercised on the phone with a class list of the three team members. With the cutoff at −75 dBm a broadcast received at −81 dBm was rejected, and with the cutoff at −90 dBm the same phone was marked present at −84 dBm and received its acknowledgement. A 180-second recording at a fixed position gave a mean RSSI of −81.3 dBm with a standard deviation of 4.0 dB and a spread from −89 to −71 dBm, which shows why the cutoff must be set from measurements in the actual room. All automated tests of the packet format and class-list parser pass.

TABLE OF CONTENTS

S.NO	TITLE	Page No.
1.	INTRODUCTION	1
2.	LITERATURE REVIEW	4
3.	PROPOSED METHODOLOGY	7
4.	RESULTS AND DISCUSSION	15
5.	CONCLUSION	21
6.	FUTURE ENHANCEMENT	22
7.	SOURCE CODE	23
	REFERENCES	27

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System architecture	7
3.2	BLE channels in the 2.4 GHz band	8
3.3	Layout of the advertising packet	9
3.4	Message exchange between the student and teacher phones	10
3.5	Flow chart of the teacher's validation	11
3.6	Log-distance path-loss model for three exponents	12
4.1	Student screen before marking	15
4.2	Student screen after the teacher's acknowledgement	15
4.3	Student screen when the signal was too weak	16
4.4	Teacher screen with the class list and manual marking	16
4.5	RSSI of the phone over 180 seconds at a fixed position	17
4.6	Distribution of the recorded RSSI values	18

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of literature survey	6
3.1	Fields of the application payload	9
3.2	Distance at which the RSSI reaches each cutoff (model)	13
3.3	Threats of proxy attendance and countermeasures	13
3.4	Modules of the system	14
3.5	Hardware and software used	14
4.1	Attendance marking with two RSSI cutoffs	17
4.2	RSSI statistics at a fixed position	18
4.3	Tests carried out	19
4.4	Comparison between existing and proposed systems	20

ABBREVIATIONS

BLE – Bluetooth Low Energy

CSV – Comma-Separated Values

dBm – Decibels relative to one milliwatt

GAP – Generic Access Profile

ID – Identifier

MAC – Media Access Control (address)

QR – Quick Response (code)

RFID – Radio-Frequency Identification

RSSI – Received Signal Strength Indicator

SDK – Software Development Kit

SIG – Special Interest Group (Bluetooth)

UI – User Interface

UTF-8 – 8-bit Unicode Transformation Format

Chapter 1
INTRODUCTION

1.1 Introduction

Attendance is recorded in almost every lecture, and in most classrooms it is still done by calling out names or passing a sheet around. For a class of sixty students a roll call takes several minutes, which adds up to hours of teaching time over a semester, and the result is only as honest as the students: one student can answer or sign for another who is absent. This practice, known as proxy attendance, is the main weakness that automated attendance systems try to remove.

Every student now carries a smartphone, and every smartphone has a Bluetooth Low Energy (BLE) radio. BLE is a short-range wireless technology that works within roughly ten to a few tens of metres indoors, and its received signal strength falls as the distance grows. These two properties make it a natural fit for attendance: a phone that can hear the teacher's phone, and that the teacher's phone can hear strongly, is very likely in the same room. This project uses that idea to build an attendance system entirely in software, on phones the teacher and students already own.

The system is a single Flutter application for Android with a teacher mode and a student mode. The teacher starts a session and the phone broadcasts a session code; each student taps one button, confirms their identity with the phone's fingerprint or face unlock, and is marked present within a few seconds. The teacher sees a live list of present and absent students and exports it as a CSV file at the end of the class.

1.2 Problems with Existing Attendance Methods

Roll call and paper sheets are slow and easy to cheat. QR-code systems speed things up, but a photo of the code can be forwarded to a student who is not in the room. RFID cards and fingerprint terminals need dedicated hardware in every classroom, and a card can be handed to a friend. Earlier Bluetooth systems identified students by the Bluetooth address of their phones, which proves only that the phone is nearby, not that its owner is; and modern phones randomise their BLE addresses and hide the hardware address from apps for privacy, which breaks this approach.

A useful system therefore has to answer three questions for each student at once: is the phone in the room, is the broadcast from this class rather than a recording of an earlier one, and is the registered student the person holding the phone. None of the common methods answers all three without extra hardware.

1.3 Objectives

- Build one Android app with a teacher mode and a student mode that exchange attendance over BLE advertisements, without pairing or connections.
- Fit every message into a single legacy BLE advertisement of 31 bytes.
- Verify the student with the phone's biometric lock before attendance is sent.
- Reject replayed broadcasts with a random code that changes every session.
- Limit marking to nearby phones with an RSSI cutoff that the teacher can adjust.
- Bind each roll number to one phone and flag any other phone that claims it.
- Let the teacher load a class list, see absent students, mark students by hand and export the result.
- Confirm attendance back to the student's phone, and measure the behaviour of RSSI on real hardware.

1.4 Scope

The system runs on Android phones with BLE advertising support and a fingerprint or face unlock; iPhones restrict custom BLE advertisements and are out of scope. Attendance is kept on the teacher's phone and exported as CSV; there is no server or shared database. Presence is judged from signal strength only, so the system decides whether a phone is close to the teacher, not its exact position in the room. A Python script that turns a Windows laptop into a teacher device is included for testing with a single phone.

1.5 Organisation of the Report

Chapter 2 reviews related work on attendance systems, BLE and RSSI-based proximity. Chapter 3 describes the proposed methodology: the architecture, the packet format, the teacher and student algorithms, the RSSI model and the protection against proxy attendance. Chapter 4 presents the results on real hardware and compares the system with existing methods.

Chapters 5 and 6 give the conclusion and future enhancements, and Chapter 7 lists the main source code.

Chapter 2
LITERATURE REVIEW

The work that shaped this project falls into three groups: the Bluetooth Low Energy standard and studies of its performance, research on using received signal strength to estimate proximity indoors, and earlier attendance systems built on QR codes, RFID, biometrics and Bluetooth. This chapter reviews each group and ends with the gap that the proposed system addresses.

The Bluetooth Core Specification published by the Bluetooth Special Interest Group [1] defines BLE, including the three primary advertising channels, the 31-byte legacy advertising payload and the manufacturer-specific data field that this project uses to carry its messages. An early evaluation of BLE by Gomez, Oller and Paradells [2] analysed its latency, throughput and energy use and concluded that BLE is well suited to applications that exchange small amounts of data occasionally, which is exactly the pattern of an attendance broadcast.

Signal strength and distance are related through the log-distance path-loss model described by Rappaport [3], in which the received power falls by 10n decibels for every tenfold increase in distance, with a path-loss exponent n of about 2 in free space and higher indoors. In practice the relation is noisy. Dong and Dargie [4] measured RSSI indoors and found it to fluctuate strongly with orientation, obstacles and multipath, making it a poor estimate of exact distance. Faragher and Harle [5] studied BLE beacons for location fingerprinting and showed that fast fading and differences between the three advertising channels cause large variations between individual readings. These findings led us to use RSSI only as a cutoff that the teacher tunes for the room, rather than to convert it into metres.

BLE proximity has nevertheless been used successfully in real rooms. A survey by Zafari, Gkelias and Leung [6] compares indoor localisation technologies and techniques and notes that BLE is attractive because it is low-cost and present in nearly all phones, while its accuracy is limited by RSSI variation. Spachos and Plataniotis [7] used BLE beacons to detect visitors near exhibits in a museum and showed that proximity-based services work with careful tuning. The Exposure Notification system designed by Apple and Google [8] went further and used phone-to-phone BLE advertisements with rotating identifiers to estimate how close two

people were, without pairing; our design borrows its connectionless, advertisement-only approach.

Identity is the second half of the attendance problem. Jain, Ross and Prabhakar [9] describe how biometric recognition ties a credential to a person rather than to something the person carries, which is why our student mode requires the phone's fingerprint or face unlock before sending attendance. Earlier attendance systems tried other credentials. Masalha and Hirzallah [10] used a QR code that students scan, which is fast but can be photographed and shared with someone outside the room. Arulogun et al. [11] used RFID cards and readers, which require hardware in every room and accept a card regardless of who holds it. Bhalla et al. [12] used the teacher's phone to discover students' phones over classic Bluetooth and matched their device addresses, which checks the phone but not the student, and which modern phones defeat by randomising or hiding their addresses.

Taken together, the surveyed systems either need extra hardware, check only a device and not the person, or cannot tell a live broadcast from a recording. None combines short-range presence, a per-session challenge and biometric confirmation on hardware the class already owns. This is the gap the proposed system addresses, and Table 2.1 summarises the studies reviewed in this chapter.

Table 2.1: **Summary of literature survey**

No.	Title	Year	Method	Inference
1	Bluetooth Core Specification v5.4 [1]	2023	Standard for BLE advertising, channels and packet formats	Defines the 31-byte legacy advertisement used here
2	Overview and evaluation of Bluetooth Low Energy [2]	2012	Analysis and measurement of BLE	BLE suits short, infrequent messages at low power
3	Wireless Communications: Principles and Practice [3]	2002	Textbook; log-distance path-loss model	Basis of our RSSI-distance model
4	Evaluation of the reliability of RSSI for indoor localization [4]	2012	Indoor RSSI experiments	RSSI fluctuates; distance estimates are unreliable
5	Location fingerprinting with BLE beacons [5]	2015	BLE RSSI fingerprinting	Fading and channel effects make single readings noisy
6	A survey of indoor localization systems and technologies [6]	2019	Survey of techniques and technologies	BLE is cheap and widely available but RSSI-based
7	BLE beacons for indoor positioning at a smart museum [7]	2020	Proximity and positioning with beacons	BLE proximity works in real rooms with tuning
8	Exposure Notification Bluetooth Specification [8]	2020	Rotating IDs in BLE advertisements	Phone-to-phone proximity without pairing
9	An introduction to biometric recognition [9]	2004	Survey of biometric systems	Biometrics tie a credential to a person
10	A students attendance system using QR code [10]	2014	QR code scanned by students	Fast, but codes can be shared remotely
11	RFID-based students attendance management system [11]	2013	RFID cards and readers	Needs readers; cards can be lent
12	Bluetooth based attendance management system [12]	2013	Teacher phone discovers student phones by address	Checks the phone, not the person

Chapter 3
PROPOSED METHODOLOGY

3.1 Overview

The system consists of one Android application, written in Flutter, that runs in a teacher mode or a student mode, and an optional Python script that lets a Windows laptop act as the teacher. The two devices talk only through BLE advertisements: each side broadcasts a small packet that any nearby phone can receive, and neither side ever opens a connection. The app uses Android's BLE and biometric APIs [13] through Flutter packages. Figure 3.1 shows the architecture.

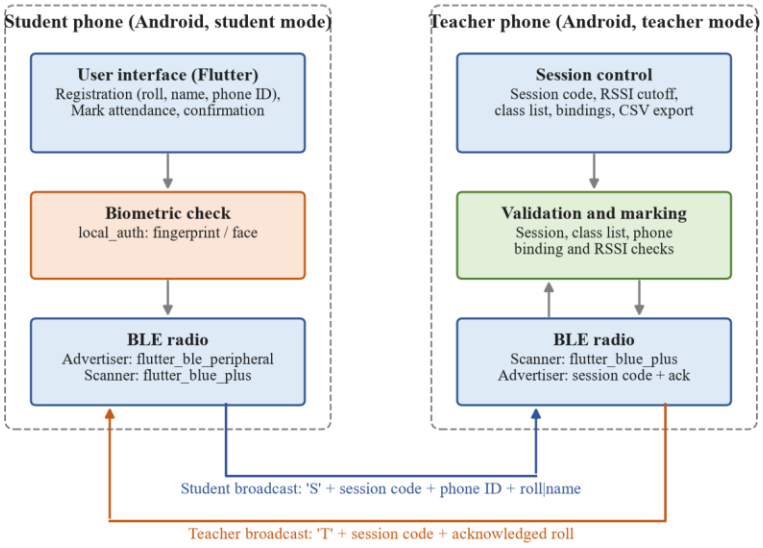


Figure 3.1: **System architecture**

Figure 3.1 divides the system into the two phones. On the student phone, the user interface registers the student once (roll number, name and a random phone ID), the biometric check confirms the student before anything is sent, and the BLE radio both advertises the student's packet and scans for the teacher's. On the teacher phone, the session control holds the session code, the RSSI cutoff, the class list and the phone bindings; the validation module checks every student packet; and the radio scans for students while advertising the session code and the acknowledgements. The two radio links at the bottom are the only communication between the devices.

3.2 Bluetooth Low Energy Background

BLE operates in the 2.4 GHz ISM band, divided into 40 channels 2 MHz apart. Three of them, channels 37, 38 and 39 at 2402, 2426 and 2480 MHz, are advertising channels, chosen to avoid the most used Wi-Fi channels; the other 37 carry data during connections [1]. Figure 3.2 shows the channel plan.

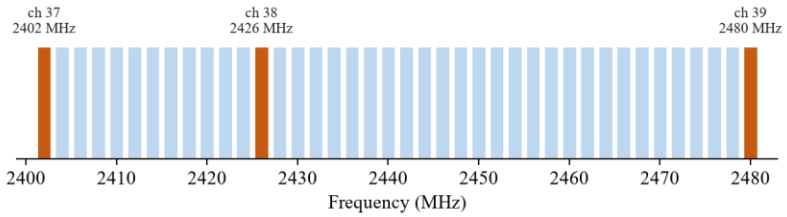


Figure 3.2: **BLE channels in the 2.4 GHz band**

A device in the advertiser role sends short advertising packets on the three advertising channels at a regular interval, and a device in the scanner role listens on them and reports each packet it hears together with its RSSI. Because advertising needs no connection, a single scanner can hear many advertisers at once, and a phone can scan and advertise at the same time. This project uses only advertising and scanning. Avoiding connections removes pairing, keeps each exchange to a few packets, and lets one teacher phone serve a whole class in parallel.

3.3 Packet Design

A legacy advertising packet carries at most 31 bytes of advertising data. The flags field uses 3 bytes, and the manufacturer-specific data structure uses a 4-byte header: a length byte, the type 0xFF and the 2-byte company identifier. We use the company identifier 0xFFFF, which the Bluetooth SIG reserves for testing. That leaves 24 bytes for the application payload, as shown in Figure 3.3.

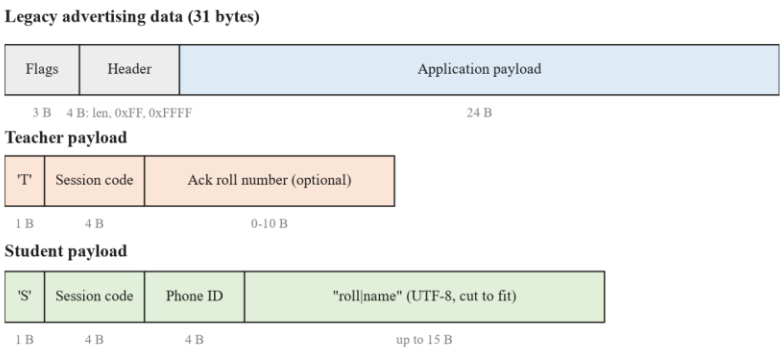


Figure 3.3: **Layout of the advertising packet**

The first byte of the payload is a tag that tells the two packet types apart. The teacher packet carries the 4-byte session code and, optionally, the roll number being acknowledged. The student packet carries the session code, a 4-byte phone ID and the text “roll|name” in UTF-8. Roll numbers are limited to 10 characters so that they always fit; the name is cut to whatever space remains. Table 3.1 lists the fields.

Table 3.1: **Fields of the application payload**

Field	Size	Packet	Purpose
Tag	1 byte	Both	‘T’ for teacher, ‘S’ for student
Session code	4 bytes	Both	Random 32-bit value, new for every session; rejects old or replayed packets
Ack roll number	0–10 bytes	Teacher	Roll number whose attendance was accepted
Phone ID	4 bytes	Student	Random value created once per install; binds a roll number to a phone
roll|name	up to 15 bytes	Student	Roll number (up to 10 characters) and as much of the name as fits

3.4 Message Exchange

Marking one student takes three broadcasts, shown in Figure 3.4. The teacher’s phone advertises the session code throughout the session. The student’s phone scans until it hears it, asks for the fingerprint or face unlock, and then advertises its own packet. The teacher’s phone validates the packet and, if it is accepted, places the student’s roll number in its advertisement as an acknowledgement.

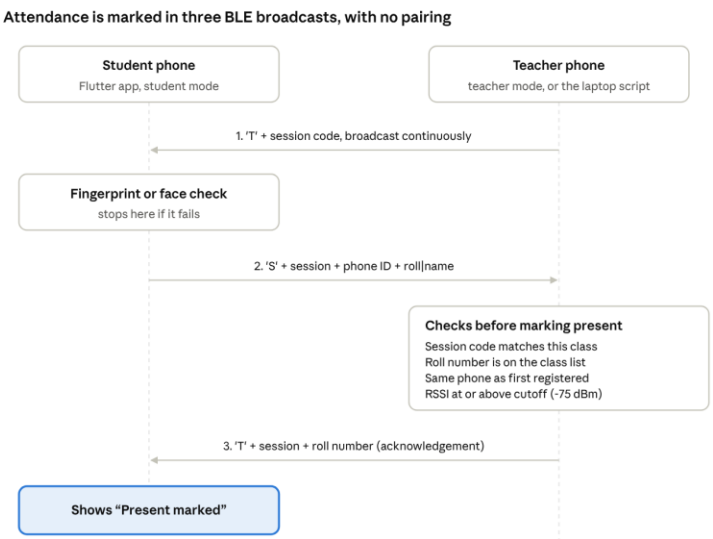


Figure 3.4: **Message exchange between the student and teacher phones**

Figure 3.4 also shows that every step is a broadcast: the arrows are advertisements that any phone in range can hear, not messages over a connection. This is what allows many students to mark attendance at the same time.

3.5 Teacher Algorithm

The teacher's phone runs Algorithm 3.1 for the whole session. Its job is to accept only students who are in range, in this session, on the class list and on their own phones, and to tell each of them that they were accepted.

Algorithm 3.1: Teacher session

Step 1: Generate a random 32-bit session code and start advertising the packet 'T' + session code.

Step 2: Start a continuous scan for advertisements with company identifier 0xFFFF.

Step 3: For each student packet, read the session code, phone ID, roll number and RSSI.

Step 4: If the session code differs from the current one, ignore the packet (stale or replayed).

Step 5: If a class list is loaded and the roll number is not on it, flag the student as "Not in class list".

Step 6: If the roll number is bound to a different phone ID, flag it as "Registered on a different phone".

Step 7: If the roll number is already present, update its RSSI and the time it was last heard.

Step 8: Otherwise, if the RSSI is at or above the cutoff, mark the student present and bind the roll number to this phone ID if it was unbound.

Step 9: Every second, place the next student still broadcasting into the acknowledgement field of the teacher packet.

Step 10: On request, mark or unmark a student by hand, allow a flagged phone, or export the list as CSV; stop when the teacher ends the session.

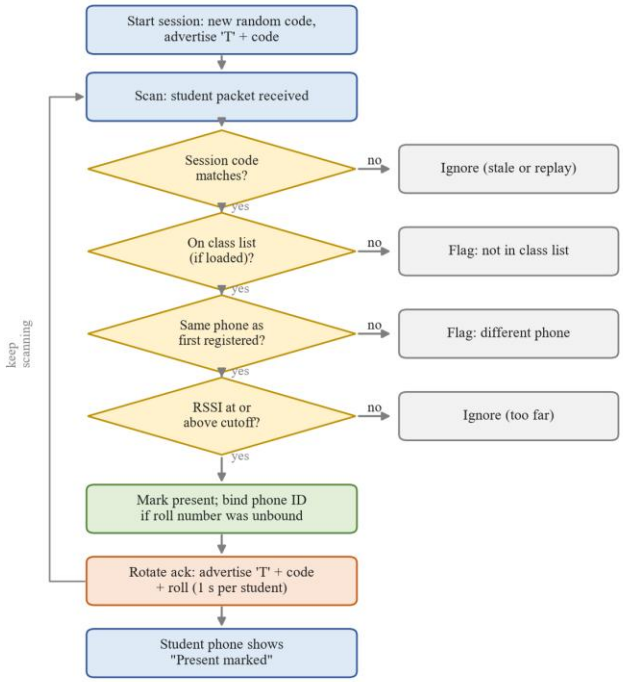


Figure 3.5: **Flow chart of the teacher's validation**

Figure 3.5 is the flow chart of Steps 3 to 9. The four checks are applied in order and each failure ends differently: a wrong session code or a weak signal is ignored silently, while a roll number missing from the class list or claimed by a different phone is flagged so that the teacher can review it and press Allow if the student has, for example, changed phones.

3.6 Student Algorithm

Algorithm 3.2: Student marking

Step 1: On first use, ask for the roll number and name, create a random 32-bit phone ID and save all three; they cannot be edited later.

Step 2: When the student taps Mark attendance, request the Bluetooth permissions and turn Bluetooth on if needed.

Step 3: Scan for a teacher packet for up to 20 seconds and read its session code; if none is found, ask the student to move closer.

Step 4: Ask for the fingerprint or face unlock; if it fails, stop.

Step 5: Advertise ‘S’ + session code + phone ID + “roll|name”.

Step 6: Scan for up to 60 seconds for a teacher packet with the same session code whose acknowledgement field equals this roll number.

Step 7: On a match, stop advertising and show “Present marked”; otherwise stop and ask the student to move closer and try again.

The student’s phone never decides that the student is present; it only reports what the teacher’s phone acknowledged. If the teacher rejects the packet for any reason, no acknowledgement arrives and the student is told to try again, which keeps the two phones consistent.

3.7 Proximity Estimation with RSSI

The power received from a transmitter falls with distance. In the log-distance path-loss model [3], the RSSI at distance d is

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}(d / d_0)$$

where RSSI(d_0) is the strength at a reference distance d_0 of 1 m and n is the path-loss exponent, about 2 in free space and between 2.7 and 4 inside buildings. Figure 3.6 plots the model for three exponents, assuming a typical phone strength of −59 dBm at 1 m, together with the two cutoffs used in our tests.

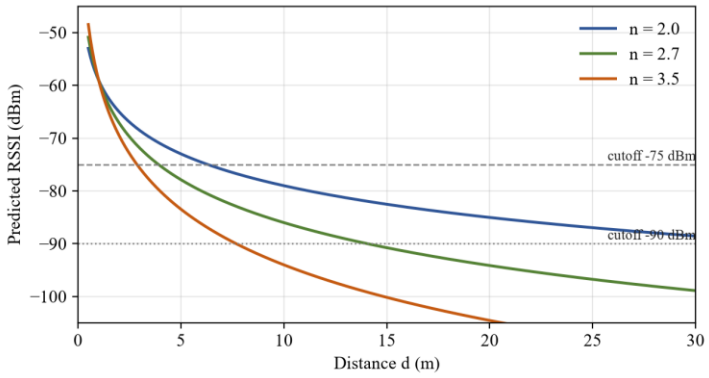


Figure 3.6: **Log-distance path-loss model for three exponents**

Solving the model for d gives the distance at which the signal reaches a cutoff, listed in Table 3.2. The numbers show why the cutoff is a setting rather than a constant: depending on the room, −75 dBm corresponds to roughly 3 to 6 m and −90 dBm to roughly 8 to 35 m. The teacher therefore adjusts the cutoff with a slider from −100 to −40 dBm, starting at −75 dBm.

Table 3.2: **Distance at which the RSSI reaches each cutoff (model)**

Cutoff	n = 2.0	n = 2.7	n = 3.5
−75 dBm	6.3 m	3.9 m	2.9 m
−90 dBm	35.5 m	14.1 m	7.7 m

3.8 Acknowledgement Rotation

The teacher packet has room for only one acknowledged roll number at a time, but many students may be waiting at once. The teacher's phone therefore keeps a list of present students that it has heard in the last 5 seconds, which means they are still broadcasting and have not yet received their acknowledgement, and every second it places the next one from this list into its advertisement. A student who receives the acknowledgement stops broadcasting, drops out of the list after 5 seconds, and so frees the slot for the others. With 30 students arriving together, each is acknowledged within about 30 seconds, well inside the 60 seconds the student's phone waits.

3.9 Protection against Proxy Attendance

Each kind of proxy attempt is met by a separate check, summarised in Table 3.3. No single check is enough on its own, but together they make proxy attendance considerably harder than with a roll call.

Table 3.3: **Threats of proxy attendance and countermeasures**

Threat	Countermeasure	Where
A friend marks you from their own phone	Roll number bound to the first phone that used it	Teacher phone
A friend carries your phone to class	Fingerprint or face unlock before sending	Student phone
Replaying a recorded broadcast	Random session code, new every session	Both phones
Marking from the corridor or hostel	RSSI cutoff set for the room	Teacher phone
A student not in this class	Class list; unknown roll numbers are flagged	Teacher phone

3.10 Modules

Table 3.4: **Modules of the system**

Module	Input	Process	Output
1 Registration	Roll number, name	Validate; create phone ID; save once	Locked profile
2 Packet codec (protocol.dart)	Session, phone ID, roll, name	Encode and parse 24-byte payloads	Advertising bytes
3 BLE layer (ble.dart)	Payload bytes	Permissions, advertise, scan with 0xFFFF filter	Payloads with RSSI
4 Student mode	Teacher packet, biometric result	Algorithm 3.2	“Present marked” or retry
5 Teacher mode	Student packets	Algorithm 3.1, ack rotation	Present, absent and flagged lists
6 Class list (roster.dart)	Pasted CSV or spreadsheet rows	Parse “roll,name” lines	Roll-to-name map
7 Export	Attendance lists	Build CSV with status and method	Shared CSV file
8 Laptop teacher (teacher.py)	Student packets	Same checks without class list	Console log and CSV

3.11 Hardware and Software Requirements

Table 3.5: **Hardware and software used**

Component	Specification
Test phone	OnePlus CPH2487, Android 16, BLE with advertising support, fingerprint sensor
Laptop	Windows 11 Home with Intel Wireless Bluetooth adapter
Mobile stack	Flutter 3.47.2, Dart 3.13.2, Android SDK
Flutter packages	flutter_blue_plus 2.3.14, flutter_ble_peripheral 3.1.0, local_auth 3.0.2, shared_preferences 2.5.6, share_plus 13.3.1
Laptop tools	Python 3.13, bleak 3.0.2 and the Windows Runtime BLE APIs
Testing	flutter_test unit tests, Python assertion tests, adb for screenshots

Chapter 4
RESULTS AND DISCUSSION

4.1 Input and Output

The application was installed on the test phone and exercised in both modes. Figures 4.1 to 4.4 are screenshots taken from the phone during the tests, with the status bar cropped.

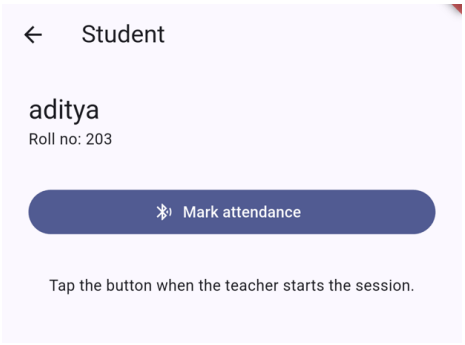


Figure 4.1: **Student screen before marking**

Figure 4.1 shows the student screen after registration. The name and roll number are fixed, and the single Mark attendance button starts Algorithm 3.2.

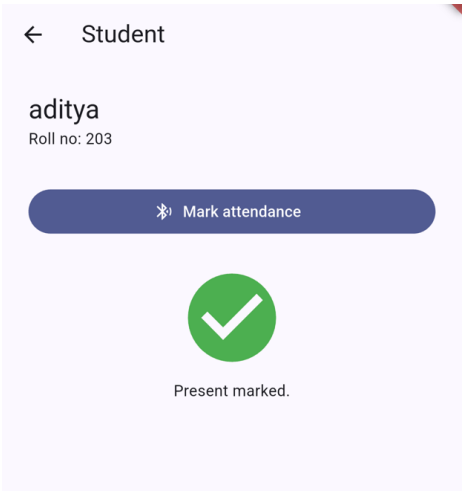


Figure 4.2: **Student screen after the teacher's acknowledgement**

Figure 4.2 shows the same screen after a successful run: the phone found the teacher, passed the fingerprint check, heard its own roll number in the teacher's acknowledgement and stopped broadcasting.

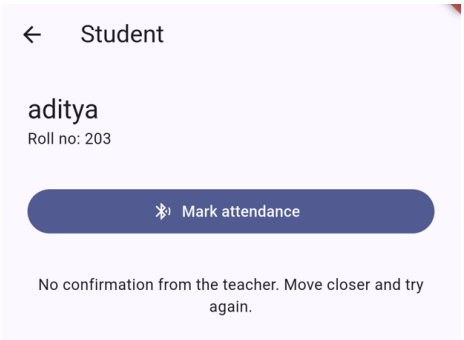


Figure 4.3: **Student screen when the signal was too weak**

Figure 4.3 shows the result when the teacher rejected the broadcast because the signal was below the cutoff. No acknowledgement arrived within 60 seconds, so the phone asked the student to move closer.

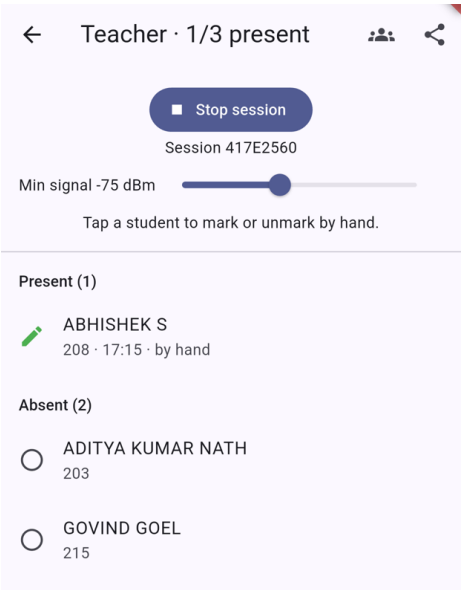


Figure 4.4: **Teacher screen with the class list and manual marking**

Figure 4.4 shows the teacher mode during a session with the class list of the three team members loaded. The title shows 1/3 present, the list is split into Present and Absent, and ABHISHEK S has been marked by tapping his name, which the screen labels “by hand” so that it can be told apart from Bluetooth marking. The slider at the top sets the RSSI cutoff.

4.2 Attendance Marking Tests

To test the full exchange with a single phone, the phone ran in student mode and the laptop ran the teacher script, which uses the same packet format and checks. The phone and the laptop

stayed in the same positions for both runs; only the cutoff was changed. Table 4.1 gives the results.

Table 4.1: **Attendance marking with two RSSI cutoffs**

Run	Cutoff	Measured RSSI	Laptop (teacher)	Phone (student)
1	−75 dBm	−81 dBm	Ignored, too weak	No confirmation (Figure 4.3)
2	−90 dBm	−84 dBm	Marked present	Present marked (Figure 4.2)

The laptop printed the following log for the two runs:

```
Session A2762173 started. Min signal -75 dBm. Ctrl+C to stop.
  ignored  203        aditya               -81 dBm (too weak)

Session F40E376B started. Min signal -90 dBm. Ctrl+C to stop.
PRESENT    203        aditya               -84 dBm
```

The two runs show the cutoff working as a range filter: the same phone in the same place was rejected at −75 dBm and accepted at −90 dBm. In the second run the phone also showed "Present marked", which confirms that the acknowledgement path works end to end. The two readings differ by 3 dB although nothing was moved, which the next section examines.

4.3 RSSI Stability at a Fixed Position

To measure how much the signal varies when nothing moves, the phone was left in teacher mode, advertising its session code, and the laptop recorded the RSSI of every advertisement it reported for 180 seconds. The laptop reported each advertisement twice, once for the packet and once for the scan response, so duplicates less than 50 ms apart were merged; one reading of −127 dBm, the value the adapter uses for an invalid measurement, was removed. Figure 4.5 shows the remaining readings over time.

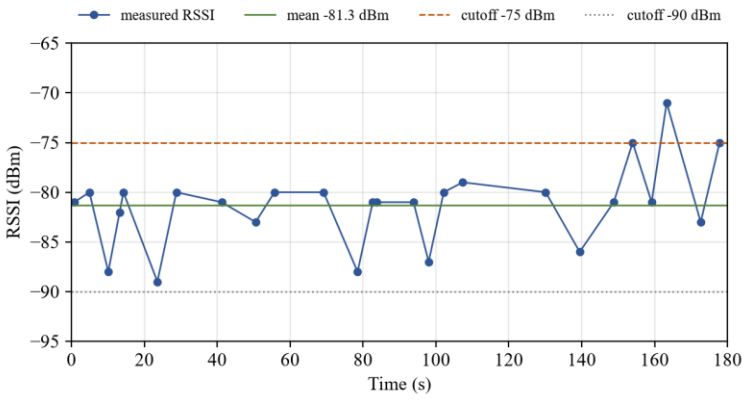


Figure 4.5: **RSSI of the phone over 180 seconds at a fixed position**

Figure 4.5 shows the RSSI moving between −89 and −71 dBm around a mean of −81.3 dBm, with no trend over time. Most readings fall between −83 and −79 dBm, but single readings jump by up to 10 dB in either direction. Figure 4.6 shows the same readings as a histogram, and Table 4.2 summarises them.

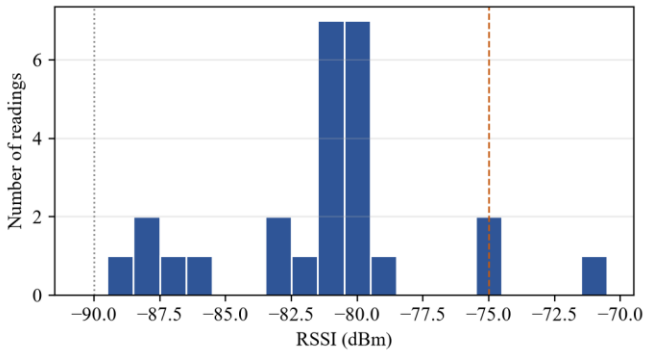


Figure 4.6: **Distribution of the recorded RSSI values**

Table 4.2: **RSSI statistics at a fixed position**

Measure	**Value**
Recording time	180 s
Advertisements reported (after merging duplicates)	27
Valid readings (after removing −127 dBm)	26
Mean RSSI	−81.3 dBm
Median RSSI	−81 dBm
Standard deviation	4.0 dB
Minimum / maximum	−89 / −71 dBm
Readings at or above −75 dBm	3 of 26 (11.5%)
Readings at or above −90 dBm	26 of 26 (100%)

Two conclusions follow. First, a single reading can differ from the average by more than the gap between two plausible cutoffs, so the cutoff must be set with a margin of several decibels above the weakest seat in the room. At this position, 11.5% of the readings would have passed a −75 dBm cutoff even though the average was 6 dB below it, and because the teacher marks a student on the first reading at or above the cutoff, a phone near the boundary can eventually be accepted if it keeps broadcasting. Averaging several readings before deciding would make the decision steadier and is listed as a future enhancement. Second, the laptop reported only 27 advertisements in 180 seconds although Android advertises several times a second in low-latency mode, because the Windows scanner passes on only a fraction of the packets it

receives. Phone-to-phone operation, which is how the system is meant to be used, receives far more packets and therefore reacts faster.

4.4 Tests Carried Out

Table 4.3 lists the automated and manual tests and their results. Each test in the table was run during the project; tests that need several phones at once are listed separately under the limitations.

Table 4.3: **Tests carried out**

No.	Test	Result
1	Teacher packet encodes and parses session code and acknowledgement	Pass (unit test)
2	Student packet round-trips and fits in 24 bytes	Pass (unit test)
3	Longest allowed roll number (10 characters) is kept in full	Pass (unit test)
4	Class list parses CSV, tab-separated rows, header and bad lines	Pass (unit test)
5	Laptop script reads the same bytes as the app	Pass (Python assertions)
6	Static analysis of the Flutter code	No issues found
7	Android build and install on the test phone	Success
8	Student marked present and acknowledged (cutoff −90 dBm)	Pass (Figure 4.2)
9	Weak signal rejected (cutoff −75 dBm, RSSI −81 dBm)	Pass (Figure 4.3)
10	Class list shows present and absent students	Pass (Figure 4.4)
11	Manual marking by tapping a student	Pass (Figure 4.4)
12	Fingerprint check required before broadcasting	Pass (prompt shown each run)

4.5 Comparison between Existing and Proposed Systems

Table 4.4 compares the proposed system with the attendance methods reviewed in Chapter 2 on the points that matter in a classroom: the hardware needed, whether presence in the room is checked, whether the person is checked, and the main way each can be defeated.

Table 4.4: **Comparison between existing and proposed systems**

Method	Extra hardware	Checks presence	Checks person	Main weakness
Roll call / sheet	None	Partly	No	Slow; easy proxy
QR code [10]	Projector or screen	No	No	Code photo can be shared
RFID cards [11]	Reader per room	Yes	No	Card can be lent
Fingerprint terminal [9]	Terminal per room	Yes	Yes	Cost; queue at the door
Bluetooth address scan [12]	None	Yes	No	Checks phone only; random addresses
Proposed system	**None**	**Yes (RSSI)**	**Yes (biometric)**	**RSSI noise; phone ID can be copied**

The proposed system is the only one in the table that checks both presence and the person without adding hardware to the classroom. Its weaknesses are the noise in RSSI, discussed in Section 4.3, and the fact that the phone ID is sent openly, discussed below.

4.6 Limitations

- A legacy advertisement leaves 24 bytes, so roll numbers are limited to 10 characters and long names are shortened; the full 15-character register number does not fit, so its last digits are used.
- RSSI varies with walls, people and how the phone is held, so the cutoff separates inside from outside only approximately.
- The phone ID is sent openly and could be copied by a modified app, and the biometric check proves only that someone enrolled on that phone unlocked it.
- The flag for a roll number claimed by a different phone, and a full class marking at the same moment, need several phones and were not tested with real devices in this project.
- iPhones restrict custom BLE advertisements, so the app runs on Android only.

Chapter 5
CONCLUSION

This project designed, built and tested a classroom attendance system based on Bluetooth Low Energy that runs entirely on the phones the teacher and students already own. The teacher and the student exchange attendance through BLE advertisements alone, without pairing or connections, and every message fits in the 24-byte payload of a single legacy advertisement. Because the exchange is connectionless, one teacher phone can serve a whole class at the same time.

The system answers the three questions that an attendance method must answer. Presence in the room is judged from the RSSI against a cutoff that the teacher sets for the room; the session is identified by a random code that changes every class, so recorded broadcasts are useless; and the person is confirmed by the phone's fingerprint or face unlock before attendance is sent, with each roll number bound to one phone. The teacher can load a class list, see absent students, approve flagged phones, mark students by hand and export the result, and every student receives a clear "Present marked" confirmation.

Tests on real hardware confirmed that the exchange works end to end and that the cutoff behaves as a range filter: the same phone was rejected at −75 dBm and accepted at −90 dBm. A three-minute recording at a fixed position showed a standard deviation of 4.0 dB and a spread of 18 dB, which explains why individual readings must not be trusted too far and why the cutoff should be set from measurements in each room with a margin.

The project therefore shows that short-range wireless communication, used carefully, can replace roll call with a faster and harder-to-cheat method at no hardware cost, while also making clear the practical limits of RSSI as a measure of distance.

Chapter 6
FUTURE ENHANCEMENT

The current system is complete for a single classroom and a single teacher phone. The following enhancements would make it more robust and easier to deploy across a department.

- **Averaged RSSI decisions.** Mark a student only when the average of several readings, rather than a single reading, is at or above the cutoff. Section 4.3 shows that this would remove most borderline acceptances.
- **Signed packets.** Derive the phone ID from a key stored in the phone's secure hardware and sign each packet with it, so that a copied ID is useless.
- **Server synchronisation.** Store class lists, phone bindings and attendance history on a server, so that they follow the teacher across devices and can be reported per student over the semester.
- **Calibration mode.** Let the teacher walk to the back of the room with a student phone and set the cutoff automatically from the measured readings.
- **Multi-receiver positioning.** Combine readings from two or three teacher devices to estimate where a phone is in the room instead of only how far it is from one device.
- **Extended advertising.** Use Bluetooth 5 extended advertising, where supported, to carry full register numbers and names.
- **iOS support.** Add an iPhone version within the limits Apple places on background BLE advertising.

Chapter 7
SOURCE CODE

The complete source is organised as one Flutter project with a tools folder for the laptop script. The files below contain the core of the system: the packet format, the BLE layer, the student's marking routine and the teacher's validation and acknowledgement logic.

7.1 protocol.dart — packet format

```
import 'dart:convert';
import 'dart:math';
import 'dart:typed_data';

/// 0xFFFF is the company ID the Bluetooth SIG reserves for testing.
const kManufacturerId = 0xFFFF;

const _teacherTag = 0x54; // 'T'
const _studentTag = 0x53; // 'S'

/// A legacy BLE advertisement is 31 bytes. Flags (3) and the manufacturer
/// header (4) leave 24 bytes for our payload.
const kMaxPayload = 24;
const kMaxRollLength = 10;

/// Random 32-bit value, used for session codes and per-install phone IDs.
int randomId() => Random.secure().nextInt(1 << 32);

/// Session code (4 bytes) + optionally the roll number being acknowledged.
/// The teacher rotates the ack through students still broadcasting, so each
/// one learns it was marked present.
Uint8List teacherPayload(int session, [String ack = '']) =>
    Uint8List.fromList([_teacherTag, ..._u32(session), ...utf8.encode(ack)]);

/// Session code (4 bytes) + phone ID (4 bytes) + "roll|name", with the name
/// cut to fit.
Uint8List studentPayload(int session, int device, String roll, String name) {
  final bytes = [
    _studentTag,
    ..._u32(session),
    ..._u32(device),
    ...utf8.encode('$roll|$name'),
  ];
  return Uint8List.fromList(bytes.take(kMaxPayload).toList());
}

({int session, String ack})? parseTeacher(List<int> d) =>
    d.length >= 5 && d[0] == _teacherTag
        ? (
            session: _readU32(d, 1),
            ack: utf8.decode(d.sublist(5), allowMalformed: true),
          )
        : null;

({int session, int device, String roll, String name})? parseStudent(
    List<int> d) {
  if (d.length < 10 || d[0] != _studentTag) return null;
  final text = utf8.decode(d.sublist(9), allowMalformed: true);
  final sep = text.indexOf('|');
  if (sep <= 0) return null;
  return (
    session: _readU32(d, 1),
    device: _readU32(d, 5),
    roll: text.substring(0, sep),
    name: text.substring(sep + 1),
  );
}
```

```
List<int> _u32(int v) => [v >> 24 & 0xFF, v >> 16 & 0xFF, v >> 8 & 0xFF, v & 0xFF];

int _readU32(List<int> d, int at) =>
    d[at] << 24 | d[at + 1] << 16 | d[at + 2] << 8 | d[at + 3];
```

7.2 ble.dart — BLE advertising and scanning

```
import 'dart:typed_data';

import 'package:flutter_ble_peripheral/flutter_ble_peripheral.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

import 'protocol.dart';

/// Asks for the advertise/connect (and, on Android 11 and older, location)
/// permissions. flutter_blue_plus asks for the scan permission itself.
Future<bool> requestBlePermissions() async {
  const refused = [
    PeripheralBluetoothState.denied,
    PeripheralBluetoothState.permanentlyDenied,
    PeripheralBluetoothState.restricted,
  ];
  if (refused.contains(await _peripheral.requestPermission())) return false;
  if (FlutterBluePlus.adapterStateNow != BluetoothAdapterState.on) {
    await FlutterBluePlus.turnOn();
  }
  return true;
}

/// Scans for our app's advertisements and reports each payload with its RSSI.
Stream<({List<int> data, int rssi})> scanPayloads() async* {
  await FlutterBluePlus.startScan(
    withMsd: [MsdFilter(kManufacturerId)],
    continuousUpdates: true,
  );
  await for (final results in FlutterBluePlus.onScanResults) {
    for (final r in results) {
      final data = r.advertisementData.manufacturerData[kManufacturerId];
      if (data != null) yield (data: data, rssi: r.rssi);
    }
  }
}

Future<void> stopScan() => FlutterBluePlus.stopScan();

final _peripheral = FlutterBlePeripheral();

Future<void> startAdvertising(Uint8List payload) => _peripheral.start(
      advertiseData: AdvertiseDataCore(
        manufacturerId: kManufacturerId,
        manufacturerData: payload,
      ),
    );

Future<void> stopAdvertising() => _peripheral.stop();
```

7.3 student_page.dart — marking attendance

```
  Future<void> _markAttendance() async {
    setState(() {
      _busy = true;
      _confirmed = false;
    });
    try {
      if (!await requestBlePermissions()) {
        return _say('Bluetooth permission denied.');
      }

      _say('Looking for the teacher’s phone…');
      int? session;
```

```
    try {
      final found = await scanPayloads()
          .map((p) => parseTeacher(p.data)?.session)
          .firstWhere((s) => s != null)
          .timeout(_findTeacherTimeout);
      session = found;
    } on TimeoutException {
      return _say('Teacher not found. Move closer and try again.');
    } finally {
      await stopScan();
    }

    _say('Teacher found. Verify it's you…');
    final ok = await LocalAuthentication().authenticate(
      localizedReason: 'Confirm you are ${widget.name}',
      biometricOnly: true,
    );
    if (!ok) return _say('Fingerprint/face check failed.');

    await startAdvertising(
      studentPayload(session!, widget.device, widget.roll, widget.name),
    );
    _say('Sending attendance… keep the app open.');
    try {
      await scanPayloads()
          .map((p) => parseTeacher(p.data))
          .firstWhere((t) => t?.session == session && t?.ack == widget.roll)
          .timeout(_broadcastTime);
      if (mounted) setState(() => _confirmed = true);
      _say('Present marked.');
    } on TimeoutException {
      _say('No confirmation from the teacher. Move closer and try again.');
    } finally {
      await stopScan();
      await stopAdvertising();
    }
  } catch (e) {
    _say('Error: $e');
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}
```

7.4 teacher_page.dart — validation and acknowledgement

```
void _onBroadcast(int session, List<int> data, int rssi) {
  final s = parseStudent(data);
  // An old or foreign session code means a replayed or stale broadcast.
  if (s == null || s.session != session) return;
  final name = _roster[s.roll] ?? s.name;
  if (_roster.isNotEmpty && !_roster.containsKey(s.roll)) {
    return _flag(s.roll, _Flag(name, s.device, 'Not in class list'));
  }
  final bound = _devices[s.roll];
  if (bound != null && bound != s.device) {
    return _flag(s.roll, _Flag(name, s.device, 'Registered on a different phone'));
  }
  final known = _present[s.roll];
  if (known != null) {
    setState(() {
      known.rssi = rssi;
      known.lastSeen = DateTime.now();
    });
  } else if (rssi >= _minRssi) {
    if (bound == null) {
      _devices[s.roll] = s.device;
      _save();
    }
    final now = DateTime.now();
    setState(() => _present[s.roll] = _Attendee(name, now, rssi: rssi, lastSeen: now));
  }
}

void _flag(String roll, _Flag flag) {
  if (!_flagged.containsKey(roll)) setState(() => _flagged[roll] = flag);
}
```

```
  /// Accepts a flagged phone for this roll number from now on (a new phone,
  /// or a student missing from the class list). Their next broadcast counts.
  void _allow(String roll) {
    setState(() {
      final f = _flagged.remove(roll)!;
      _devices[roll] = f.device;
      if (_roster.isNotEmpty) _roster.putIfAbsent(roll, () => f.name);
    });
    _save();
  }

  Future<void> _rotateAck(int session) async {
    final now = DateTime.now();
    final waiting = _present.entries
        .where((e) {
          final seen = e.value.lastSeen;
          return seen != null && now.difference(seen) < _ackWindow;
        })
        .map((e) => e.key)
        .toList()
      ..sort();
    final ack = waiting.isEmpty ? '' : waiting[_ackTurn++ % waiting.length];
    if (ack == _ack) return;
    _ack = ack;
    await stopAdvertising();
    if (_session != session) return; // stopped meanwhile
    await startAdvertising(teacherPayload(session, ack));
  }
```

7.5 roster.dart — class list parser

```
import 'dart:convert';

/// Parses one "roll,name" per line, as pasted from a CSV, notes app or
/// spreadsheet (tab-separated). Header, blank and malformed lines are skipped.
Map<String, String> parseRoster(String text) {
  final roster = <String, String>{};
  for (final line in const LineSplitter().convert(text)) {
    final sep = line.indexOf(RegExp(r'[,\t]'));
    if (sep <= 0) continue;
    final roll = line.substring(0, sep).trim();
    final name = line.substring(sep + 1).trim();
    if (roll.isEmpty || name.isEmpty || roll.toLowerCase() == 'roll') continue;
    roster[roll] = name;
  }
  return roster;
}

String formatRoster(Map<String, String> roster) =>
    roster.entries.map((e) => '${e.key},${e.value}').join('\n');
```

REFERENCES

[1] Bluetooth SIG, “Bluetooth Core Specification, Version 5.4,” Bluetooth Special Interest Group, 2023.

[2] C. Gomez, J. Oller, and J. Paradells, “Overview and evaluation of Bluetooth Low Energy: An emerging low-power wireless technology,” Sensors, vol. 12, no. 9, pp. 11734–11753, 2012.

[3] T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[4] Q. Dong and W. Dargie, “Evaluation of the reliability of RSSI for indoor localization,” in Proc. Int. Conf. Wireless Communications in Underground and Confined Areas (ICWCUCA), 2012.

[5] R. Faragher and R. Harle, “Location fingerprinting with Bluetooth Low Energy beacons,” IEEE J. Sel. Areas Commun., vol. 33, no. 11, pp. 2418–2428, 2015.

[6] F. Zafari, A. Gkelias, and K. K. Leung, “A survey of indoor localization systems and technologies,” IEEE Commun. Surveys Tuts., vol. 21, no. 3, pp. 2568–2599, 2019.

[7] P. Spachos and K. N. Plataniotis, “BLE beacons for indoor positioning at an interactive IoT-based smart museum,” IEEE Syst. J., vol. 14, no. 3, 2020.

[8] Apple and Google, “Exposure Notification – Bluetooth Specification,” v1.2, Apr. 2020.

[9] A. K. Jain, A. Ross, and S. Prabhakar, “An introduction to biometric recognition,” IEEE Trans. Circuits Syst. Video Technol., vol. 14, no. 1, pp. 4–20, 2004.

[10] F. Masalha and N. Hirzallah, “A students attendance system using QR code,” Int. J. Adv. Comput. Sci. Appl., vol. 5, no. 3, 2014.

[11] O. T. Arulogun, A. Olatunbosun, O. A. Fakolujo, and O. M. Olaniyi, “RFID-based students attendance management system,” Int. J. Sci. Eng. Res., vol. 4, no. 2, 2013.

[12] V. Bhalla, T. Singla, A. Gahlot, and V. Gupta, “Bluetooth based attendance management system,” Int. J. Innov. Eng. Technol., vol. 3, no. 1, 2013.

[13] Android Developers, “Bluetooth Low Energy overview” and “Show a biometric authentication dialog,” developer.android.com, accessed Oct. 2026.